

Custom Events Parsing, Replay Features, and Business Signals

Date: June 12, 2026 Document Version: 7.9 (Hierarchy Alignment Quality Thresholds)

Executive Summary

This technical specification outlines the ingestion, redaction, compaction, and curation pipeline for the Rejourney Research Lake. The lake serves as the unified training repository for session-level customer lifetime value (LTV) models, funnel conversion propensities, and cohort churn prediction. Native view hierarchy layouts and raw interaction event logs are continuously collected, anonymized, and exported to support multimodal pretraining. Data is ingested, cleaned, and partitioned into three distinct datasets based on privacy parameters and layout modalities:

- 1. Interaction Dataset - Mobile UI: Contains native mobile screens downsampled into pixel feature matrices (luma, edge, HSL color) alongside sparse view hierarchies.
- 2. Interaction Dataset - Web DOM: Captures web session DOM mutations via rrweb-compatible schemas, avoiding rasterization and preserving DOM hierarchy.
- 3. Behavioral Outcomes Dataset: Contains analytics clickstreams, bounce logs, and conversion label facts for baseline outcomes calibration.

Multimodal Layout Ingestion Split

The ingestion engine separates visual session replays from non-visual clickstream traffic. For native mobile applications, the platform collects the UI view hierarchy tree structure at regular layout intervals (timing check-ins) to map coordinate interactions to semantic views. For web applications, the platform captures DOM mutation trees. Non-visual sessions are claimed by the behavioral outcomes lane to prevent dataset selection bias.

1.1 Multi-Lane Ingestion Pipeline Overview

To capture session interactions at high fidelity without introducing selection bias, the extraction pipeline splits ingestion into concurrent lanes. Concurrency is throttled to prevent downstream S3 write bottlenecks, backed by a unique database lock on (session_id, lake_type) to prevent ingestion races:

Ingestion Lanes

- interaction Ingestion Lane: Claims sessions containing visual artifacts (screenshots or rrweb). It extracts visual layout frames, skeleton view hierarchies, and downsampled grids.
- behavioral_outcomes Ingestion Lane: Claims bounce, observe-only, or quota-exhausted sessions containing clickstream events. It captures event lists, labels, and stability facts without visual structures.

BullMQ Extraction Configuration

```
queue_name: 'rj-research-lake-extraction'  
concurrency_limit: 64  
max_retry_attempts: 3  
backoff_factor: exponential (delay = 1000 * 2^attempt)  
database_locking: 'SELECT FOR UPDATE' on session_id + lake_type
```

1.2 Storage Directory Trees (Raw and Curated)

The raw landing zone writes gzip-compressed JSON Lines and manifest files directly to S3 partitioned by lake type, project key, date, and sample key. A sibling ZIP archive containing uncompressed files resolves client decompression errors.

```
v1/  
  lake=interaction/project_key={project_hash}/date={yyyy-mm-dd}/sample_key={sample_hash}/  
    manifest.json  
    quality.json  
    interactions.jsonl.gz  
    ui_frames.jsonl.gz  
    ui_skeleton.jsonl.gz  
    sample_key=...zip  
  lake=behavioral_outcomes/project_key={project_hash}/date={yyyy-mm-dd}/sample_key={sample_hash}/  
    manifest.json  
    quality.json  
    events.jsonl.gz  
    session_metrics.json  
    labels.json  
    sample_key=...zip
```

The compactor pipeline merges raw S3 files into daily curated Parquet partitions under `v1_curated/` to eliminate metadata query overhead:

```
v1_curated/  
  source_lake=interaction/table=session_fact/date={dt}/platform={platform}/  
  source_lake=interaction/table=event_fact/date={dt}/event_family={family}/  
  source_lake=interaction/table=ui_frame_fact/date={dt}/platform={platform}/  
  source_lake=interaction/table=ui_skeleton_fact/date={dt}/platform={platform}/  
  source_lake=combined/table=event_fact/date={dt}/event_family={family}/
```

1.3 GDPR Anonymization, Bucketing, & Geolocation

Strict GDPR Redaction of Sensitive Identifiers

To enforce compliance with GDPR, the export pipeline completely removes and redacts all user, transaction, subscription, coupon, and refund identifiers. Stable internal user identity display IDs and session tracking keys are completely omitted, eliminating brute-force re-identification risk.

Numeric Bucketing for Funnel Properties

- Money values: cart and transaction values are quantized to the nearest \$10 up to \$200, then \$50 up to \$1,000.
- Item count changes: quantized to the nearest 5 (e.g., quantity 7 maps to item_count_change = 5).
- Touch locations: coordinates use 0..1000 normalized buckets with 5-unit step quantization.

Geolocation Integration & Safety Thresholds

Country-level codes are exported directly (low risk). Geolocation city names are exported only if the city has an active session count exceeding a safety k-anonymity threshold ($k \geq 50$) within a 24-hour window to prevent re-identification of users in low-population areas.

2.1 Interaction Dataset: Mobile UI Downsampling & Event Transitions

The interaction lane keeps visual replay structure and event rows together. Mobile screenshots become luma/edge/color grids; native hierarchy and web rweb rows remain separate modality families. Custom event names are used to populate funnel_transition beside screen_key, target_key, normalized coordinates, grid cells, and business keys.

Interaction row fields touched by event mapping

- event name source: name, type, eventName, or event_name.
- transition output: funnel_transition, used by skeleton roles, label facts, and curated event_fact rows.
- properties used with transitions: amount/value/price/cartValue/cart_value, quantity/qty, productId/product_id/sku, planId/plan_id/plan, priceId/price_id, currency, paymentProvider/payment_provider, isRenewal/is_renewal, and isTrialConversion/is_trial_conversion.

Config overrides checked first

customEventConfig field	transition
revenueEventName	purchase_complete
conversionEventName	purchase_complete
checkoutEventName	checkout_start
subscriptionStartedEventName	checkout_start
subscriberEventName	cart_add

Worker-recognized custom event names

Source of truth: backend/src/services/researchLake.ts getFunnelTransition. The worker reads event.name, event.type, event.eventName, or event.event_name, then normalizes camelCase, spaces, dashes, and dots to snake_case before exact alias matching. Project customEventConfig is checked before the defaults below.

funnel_transition	recognized event names
purchase_complete	purchase_completed, purchase_complete, purchase_success, purchase_successful, purchase_succeeded, purchase_paid, purchase_order_completed, order_complete, order_success, order_succeeded, order_paid, complete_purchase, completed_purchase, successful_purchase, conversion, conversion_complete, conversion_completed, payment_success, payment_successful, payment_succeeded, payment_completed, checkout_completed, checkout_complete, checkout_success, checkout_succeeded, booking_completed, booking_success, reservation_completed, reservation_success
checkout_start	checkout_started, checkout_start, begin_checkout, checkout_begin, start_checkout, checkout_initiated, initiate_checkout, initiated_checkout, checkout_opened, checkout_viewed, checkout_page_viewed, checkout_step_started, payment_started
cart_add	add_to_cart, cart_add, addtocart, added_to_cart, product_added_to_cart, item_added_to_cart, item_add_to_cart, cart_item_added, add_cart, add_to_basket, basket_add, added_to_basket, add_to_bag, bag_add, added_to_bag, add_to_order, item_added, product_added
product_view	view_item, product_view, viewproduct, product_viewed, view_product, product_detail_view, product_details_view, product_detail_viewed, product_details_viewed, product_opened, item_view, item_viewed, view_item_detail, listing_view, listing_viewed, content_view, content_viewed
signup	signup, sign_up, register, account_created, signup_completed, sign_up_completed, signed_up, user_signup, user_signed_up, user_registered, registration_completed, registration_complete, register_completed, account_signup, create_account, account_creation, account_created_success, signup_success, signup_started, registration_started
login	login, sign_in, signin, signed_in, logged_in, user_login, user_logged_in, sign_in_success, login_success, login_completed
paywall_view	paywall_view, paywall_exposure, view_paywall, paywall_viewed, paywall_seen, paywall_opened, paywall_shown, paywall_displayed, paywall_impression, subscription_paywall_viewed
plan_selected	plan_selected, pricing_plan_selected, select_plan, pricing_viewed, pricing_page_viewed, pricing_page_view, selected_plan, plan_chosen, choose_plan, chose_plan, subscription_plan_selected, package_selected, product_package_selected, price_selected, tier_selected

funnel_transition	recognized event names
coupon_use	coupon_use, coupon_used, apply_coupon, coupon_applied, coupon_apply, discount_applied, apply_discount, promo_code_applied, promo_applied, promo_code_used, discount_code_applied, voucher_applied
trial_start	trial_started, trial_start, begin_trial, start_trial, trial_begin, free_trial_started, free_trial_start, trial_activated, trial_began, trial_created
subscription_start	subscription_started, subscription_start, subscribe, subscribed, subscription_created, subscription_activated, subscription_success, subscription_successful, subscription_succeeded, subscription_purchased, plan_started, plan_activated, membership_started, membership_created
refund	refund, refund_completed, refunded, refund_processed, refund_issued, refund_success, refund_succeeded, refund_created, order_refunded, payment_refunded
cancellation	cancel, canceled, cancelled, cancel_subscription, cancellation, subscription_cancelled, subscription_cancelled, subscription_cancel, subscription_cancel_requested, subscription_ended, subscription_expired, plan_cancelled, plan_canceled, unsubscribe, unsubscribed, membership_cancelled, membership_canceled
payment_failure	payment_failure, payment_failed, charge_failed, payment_error, payment_declined, card_declined, transaction_failed, checkout_failed, purchase_failed, payment_unsuccessful, invoice_payment_failed, billing_error, billing_failed
onboarding_completed	onboarding_completed, onboarding_milestone, complete_onboarding, onboarding_complete, onboarding_finished, onboarding_finish, onboarding_done, onboarding_success, onboarding_step_completed, tutorial_completed, setup_completed, activation_completed
feature_used	feature_used, key_feature_used, use_feature, feature_use, used_feature, feature_clicked, feature_activated, feature_enabled, key_feature_activated, tool_used, action_used, integration_connected, workspace_created

2.1 Addendum: Expanded Worker-Recognized Custom Events

This addendum extends the worker-recognized custom event names in section 2.1. The change is additive: existing lake objects do not need to be restarted, rewritten, or backfilled. Newly collected sessions that emit these names populate `funnel_transition`, `event_fact`, and `lifecycle_events_present` using the same normalization rules already described in the report.

Normalization remains exact after converting camelCase, spaces, dashes, and dots to snake_case. Project `customEventConfig` still takes precedence over the defaults below.

Ad exposure and click transitions

funnel_transition	recognized event names
ad_viewed	ad_viewed, ad_view, ad_seen, ad_shown, ad_displayed, ad_impression, ad_exposure, ad_rendered
ad_clicked	ad_clicked, ad_click, ad_tapped, ad_tap, ad_cta_clicked, ad_opened
native_ad_viewed	native_ad_viewed, native_ad_impression, native_ad_seen, native_ad_shown, native_ad_displayed
native_ad_clicked	native_ad_clicked, native_ad_click, native_ad_tapped, native_ad_cta_clicked
banner_ad_viewed	banner_ad_viewed, banner_ad_impression, banner_ad_seen, banner_ad_shown, banner_ad_displayed
banner_ad_clicked	banner_ad_clicked, banner_ad_click, banner_ad_tapped, banner_ad_cta_clicked
interstitial_ad_viewed	interstitial_ad_viewed, interstitial_ad_impression, interstitial_ad_seen, interstitial_ad_shown, interstitial_ad_displayed
interstitial_ad_clicked	interstitial_ad_clicked, interstitial_ad_click, interstitial_ad_tapped, interstitial_ad_cta_clicked
rewarded_ad_viewed	rewarded_ad_viewed, rewarded_ad_impression, rewarded_ad_seen, rewarded_ad_shown, rewarded_video_ad_viewed, rewarded_video_ad_impression
rewarded_ad_clicked	rewarded_ad_clicked, rewarded_ad_click, rewarded_ad_tapped, rewarded_video_ad_clicked
video_ad_viewed	video_ad_viewed, video_ad_impression, video_ad_seen, video_ad_shown, video_ad_started
video_ad_clicked	video_ad_clicked, video_ad_click, video_ad_tapped, video_ad_cta_clicked
sponsored_ad_viewed	sponsored_ad_viewed, sponsored_ad_impression, sponsored_ad_seen, sponsored_ad_shown, sponsored_content_viewed
sponsored_ad_clicked	sponsored_ad_clicked, sponsored_ad_click, sponsored_ad_tapped, sponsored_content_clicked

2.1 Addendum: Expanded Worker-Recognized Custom Events

The following 15 event groups were selected for product choice, product confidence, sharing, upgrade, discount, and cancellation-flow research. They are intended to make funnel and flow studies usable even before a purchase, subscription, or cancellation outcome is observed.

Product, subscription, and cancellation-flow transitions

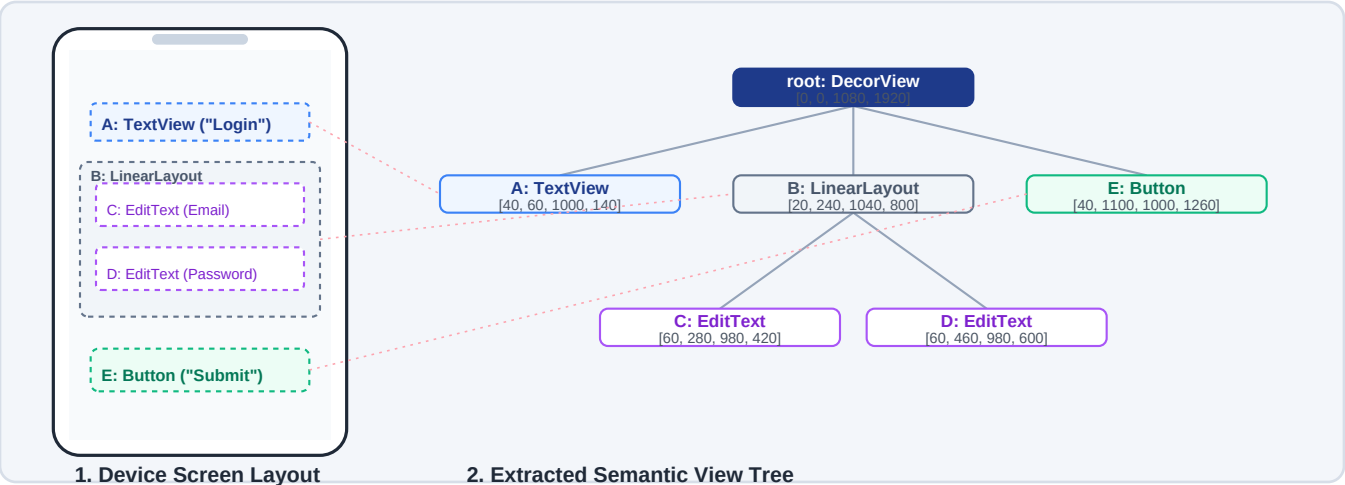
funnel_transition	recognized event names
variant_selected	variant_selected, product_variant_selected, select_variant, variant_chosen
size_selected	size_selected, product_size_selected, select_size, size_chosen
color_selected	color_selected, product_color_selected, select_color, color_chosen
quantity_changed	quantity_changed, quantity_updated, item_quantity_changed, cart_quantity_changed
reviews_viewed	reviews_viewed, review_viewed, product_reviews_viewed, ratings_viewed
shipping_info_viewed	shipping_info_viewed, shipping_viewed, shipping_details_viewed, delivery_info_viewed
return_policy_viewed	return_policy_viewed, returns_viewed, returns_policy_viewed, refund_policy_viewed
wishlist_add	wishlist_add, add_to_wishlist, added_to_wishlist, product_added_to_wishlist, favorite_added, saved_item
share_clicked	share_clicked, share_click, share_tapped, product_shared, item_shared
upgrade_clicked	upgrade_clicked, upgrade_click, upgrade_cta_clicked, click_upgrade
upgrade_completed	upgrade_completed, upgrade_complete, upgraded, plan_upgraded, subscription_upgraded
discount_offer_viewed	discount_offer_viewed, discount_viewed, offer_viewed, promo_offer_viewed
discount_offer_accepted	discount_offer_accepted, discount_accepted, offer_accepted, promo_offer_accepted
cancel_flow_started	cancel_flow_started, cancellation_started, cancel_started, subscription_cancel_flow_started
cancel_confirmed	cancel_confirmed, cancellation_confirmed, subscription_cancel_confirmed, subscription_cancellation_confirmed

2.2 Native Mobile View Hierarchy Snapshots & Tree Structure

A native view hierarchy is a logical tree representation of all UI elements (views/widgets) currently rendered on the screen. Unlike rasterized screenshots, it captures the semantic identity, relative nesting, and pixel bounding boxes of elements directly from the OS window manager. This structure is logged in ui_skeleton.jsonl.gz with an observed cadence profile so training can distinguish screenshot-aligned hierarchy from sparse timeline context:

Field	Meaning / Value
source_kind=hierarchy	Identifies native mobile hierarchy event records.
visual_modality=mobile_hierarchy_snapshot	Routes the hierarchy metadata to native layout encoders.
hierarchy_snapshot_sparse	Boolean: false for screenshot-aligned hierarchy, true for sparse timeline hierarchy.
role/type_family	Masked native widgets: control, input, search_input, map, media, list, text, container.

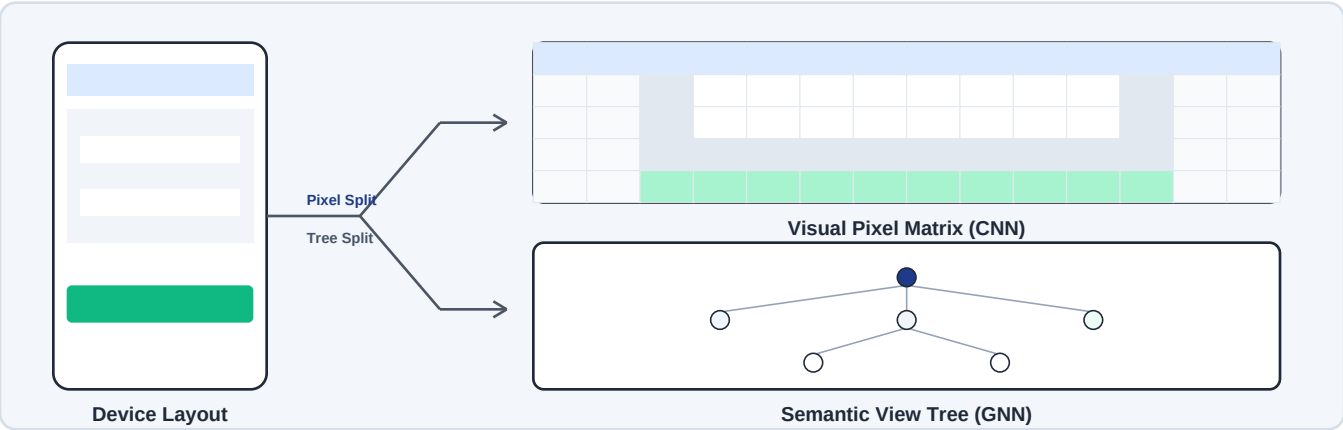
The diagram below illustrates how a visual mobile screen (1) translates into an extracted tree (2). Mapping lines connect screen elements directly to their respective logical nodes:



In practice, hierarchy cadence can vary by package version: every visual frame, every second visual frame, or every few seconds. During training, the mobile_hierarchy_encoder maps parent-child relationships using a Graph Neural Network (GNN), while capture_profile.hierarchy decides whether those nodes can be joined to screenshot frames or only used as sparse semantic context.

2.3 View Hierarchy vs. Pixel Matrix: Semantics & Use Cases

To build robust predictive models, layouts must represent not just visual patterns but structural components. The pipeline splits mobile interaction samples into a spatial Pixel Feature Grid and a semantic View Tree, serving different downstream model needs:



Comparison Matrix & Operational Use Cases

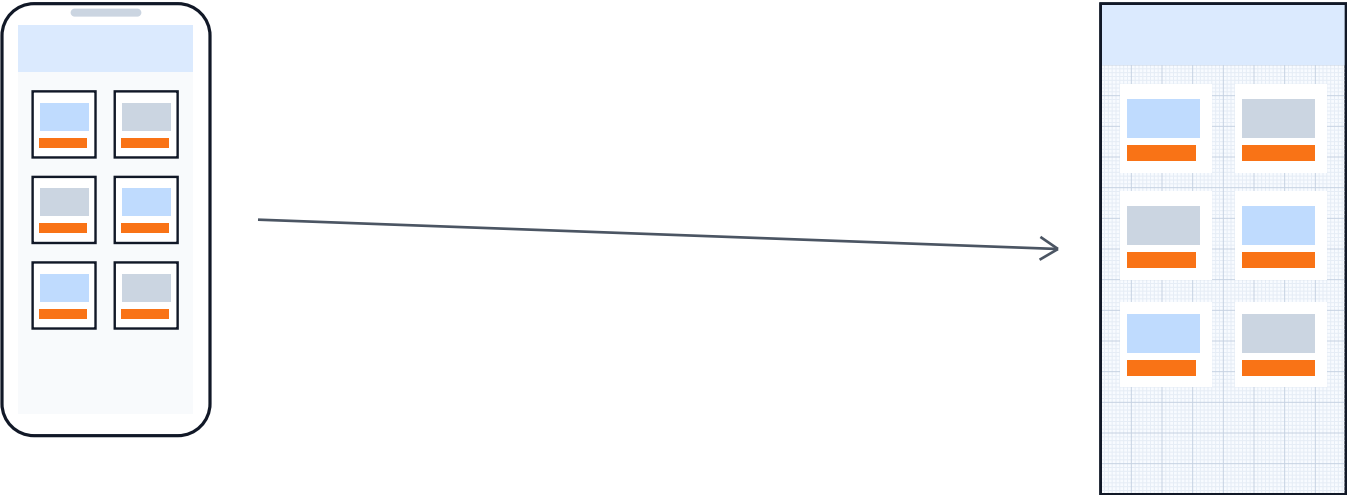
Dimension / Feature	Visual Pixel Matrix (CNN)	Semantic View Hierarchy (GNN)
Core Concept	Downsampled 2D grid of luma, edge, and color features.	Logical tree of UI widgets with type roles and nested bounds.
Downstream ML	Processed via CNNs/Vision models for spatial density.	Processed via GNNs/Transformers for structural paths.
PII Redaction	Anonymized by blurring/dropping raw screen pixels.	Inherent immunity by masking text and retaining roles.
Device Scale	Vulnerable to aspect ratio shifts and DPI differences.	Scale-invariant coordinates and topology-based structures.
Primary Use	Analyzing visual complexity and layout changes.	Mapping taps to nodes and predicting logical user intent.

By utilizing this dual-encoder approach, Rejourney's pipeline captures both what the interface looks like and what it actually does. The Pixel Matrix helps models identify visual roadblocks (e.g. cluttered screens), while the View Tree translates raw touch coordinates into semantic actions (e.g. clicking 'Submit') irrespective of device screen dimensions.

2.4 Phone UI Portrait Mockups: Product Feed and Checkout Form

The phone mockups below are drawn from the same deterministic element boxes used to render each matrix. This makes the UI-to-grid mapping inspectable and repeatable.

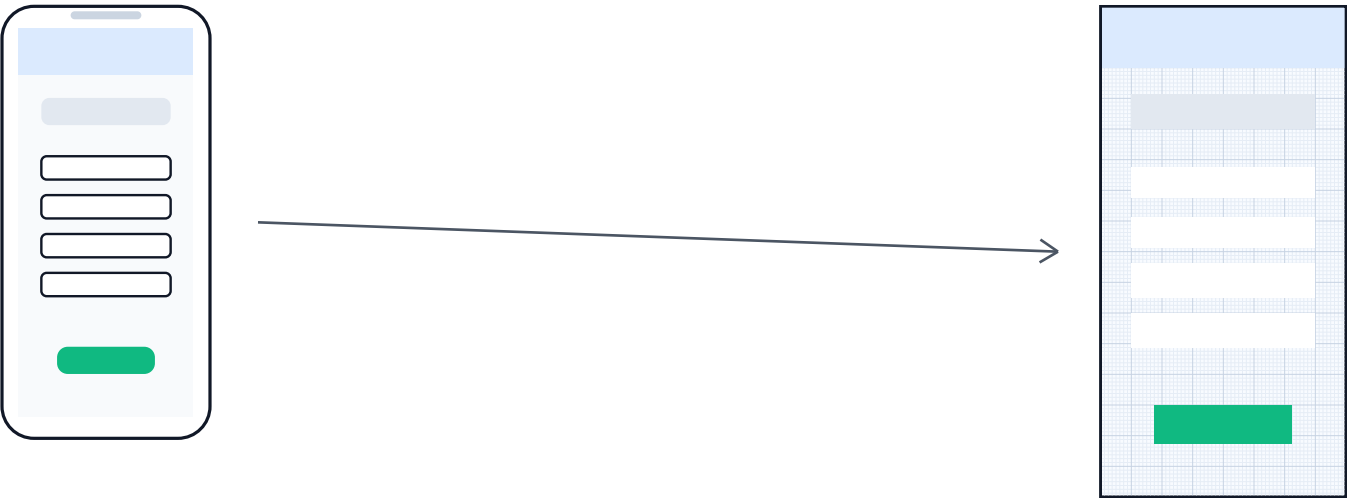
Phone Portrait Product Feed -> 64x128



Product card boxes, image blocks, and CTA strips are quantized from the same normalized element list.

64 columns x 128 rows. Product card boxes, image blocks, and CTA strips are quantized from the same normalized element list.

Phone Portrait Checkout/Form -> 64x128



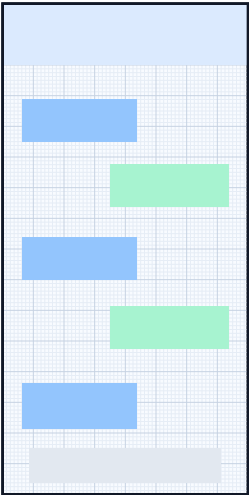
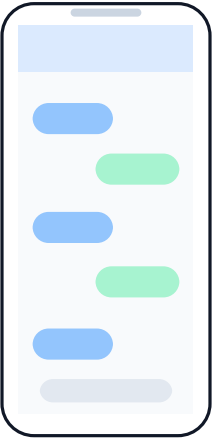
Input fields and submit control occupy exact horizontal cell bands, preserving checkout structure without text.

64 columns x 128 rows. Input fields and submit control occupy exact horizontal cell bands, preserving checkout structure without text.

2.5 Phone UI Portrait Mockups: Chat Messaging and Dashboard Metrics

These examples show how conversational layouts and finance widgets become stable 64x128 feature grids without raw text or private values.

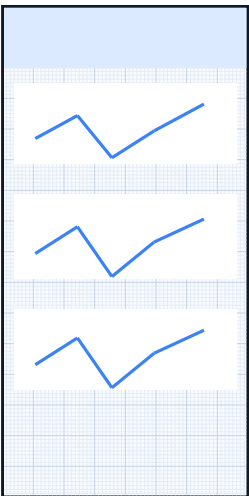
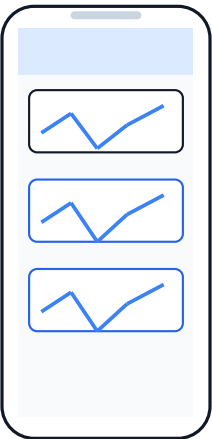
Phone Portrait Chat/Messaging List -> 64x128



Alternating bubbles map to left/right cell spans, so conversation rhythm is visible without message text.

64 columns x 128 rows. Alternating bubbles map to left/right cell spans, so conversation rhythm is visible without message text.

Phone Portrait Dashboard/Stock Metrics -> 64x128



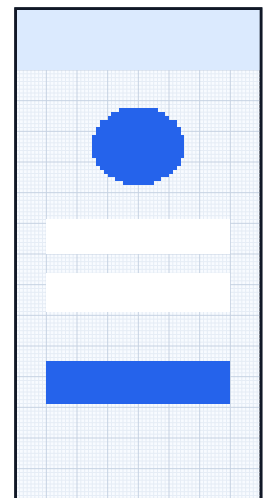
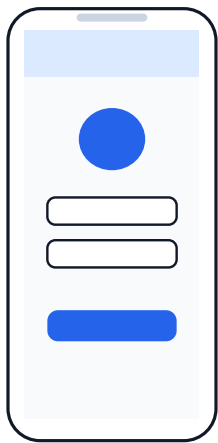
Card bounds and sparkline vectors are quantized from the same coordinates, keeping chart direction deterministic.

64 columns x 128 rows. Card bounds and sparkline vectors are quantized from the same coordinates, keeping chart direction deterministic.

2.6 Phone Mockups: Login Welcome Screen and Landscape Map

Portrait and landscape screens keep different dimensions so the matrix follows the viewport orientation instead of stretching content.

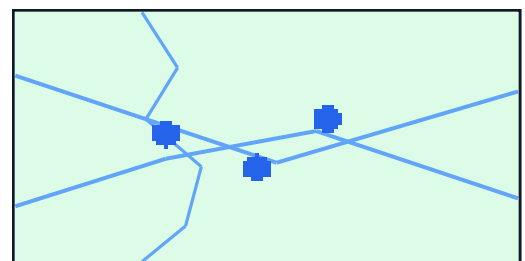
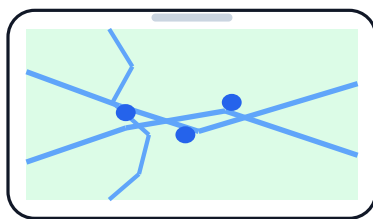
Phone Portrait Login/Welcome Screen -> 64x128



64 columns x 128 rows. Logo, fields, and CTA are cell-aligned from identical geometry, so the matrix mirrors the login layout.

Logo, fields, and CTA are cell-aligned from identical geometry, so the matrix mirrors the login layout.

Phone Landscape Map/Radar -> 128x64



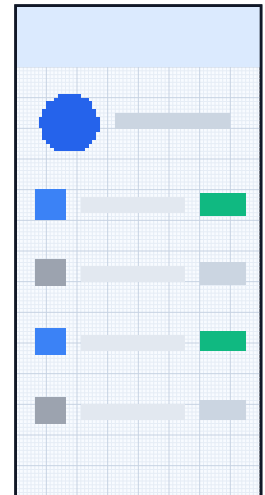
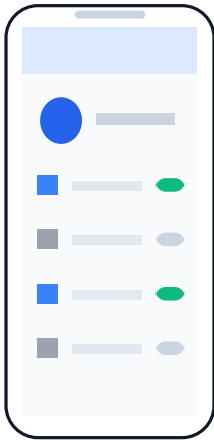
128 columns x 64 rows. Map lines and visible markers are encoded as edge cells; imagery and labels are discarded.

Map lines and visible markers are encoded as edge cells; imagery and labels are discarded.

2.7 Phone UI Portrait Mockups: Settings Page and Search/Filter

Settings and search pages demonstrate repeated row structures, toggles, chips, and result cards mapped into deterministic cells.

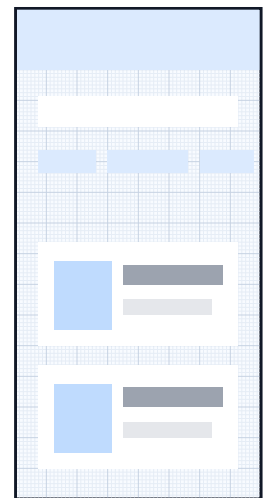
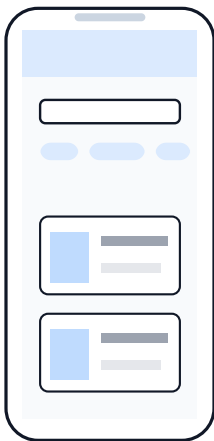
Phone Portrait Settings Page -> 64x128



64 columns x 128 rows.
Rows, icons, and toggle states occupy repeatable cell bands for preference and settings-flow modeling.

Rows, icons, and toggle states occupy repeatable cell bands for preference and settings-flow modeling.

Phone Portrait Search/Filter Page -> 64x128



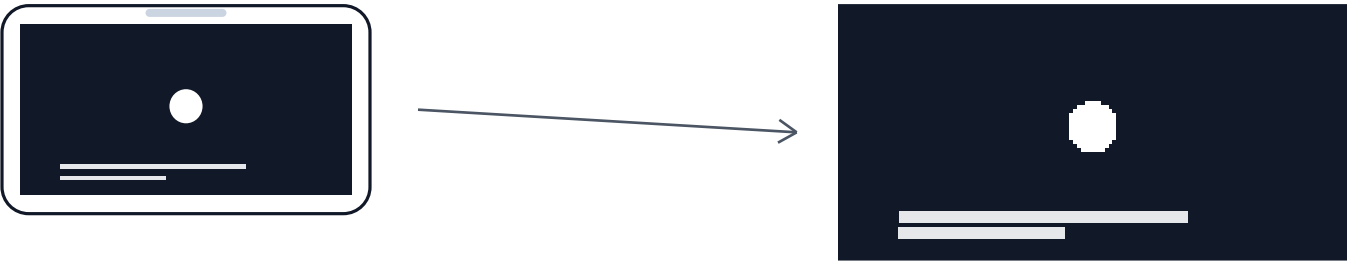
64 columns x 128 rows.
Search input, filter chips, and result cards are copied into the grid as exact element-cell regions.

Search input, filter chips, and result cards are copied into the grid as exact element-cell regions.

2.8 Phone Landscape Mockups: Video Player and Dashboard

Widescreen layouts use 128x64 matrices, preserving horizontal structure for media, maps, and dashboard surfaces.

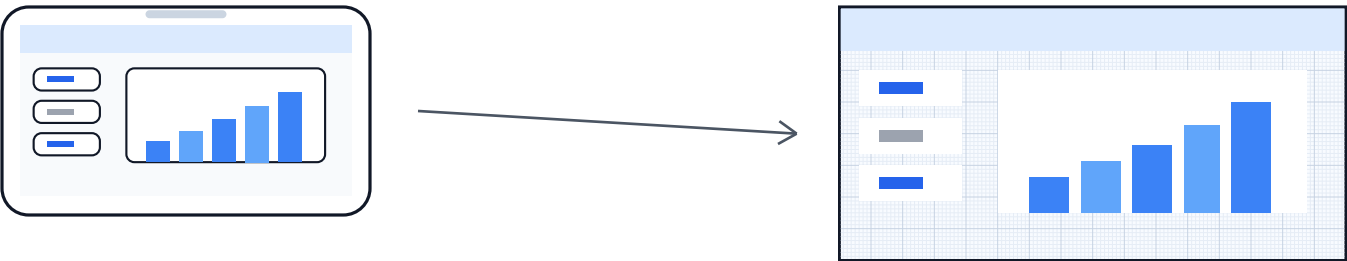
Phone Landscape Video Player -> 128x64



128 columns x 64 rows. Media area, play affordance, and subtitle bars map to deterministic cells while frame imagery is abstracted.

Media area, play affordance, and subtitle bars map to deterministic cells while frame imagery is abstracted.

Phone Landscape Dashboard -> 128x64



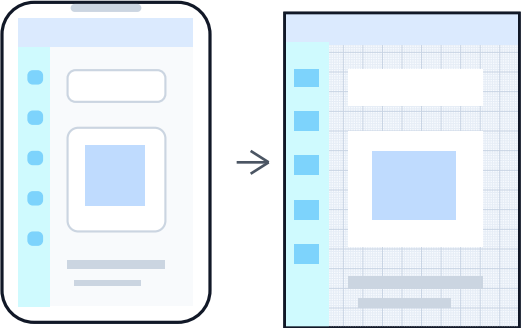
128 columns x 64 rows. Left KPI tiles and right chart bars retain their relative positions in the 128x64 landscape tensor.

Left KPI tiles and right chart bars retain their relative positions in the 128x64 landscape tensor.

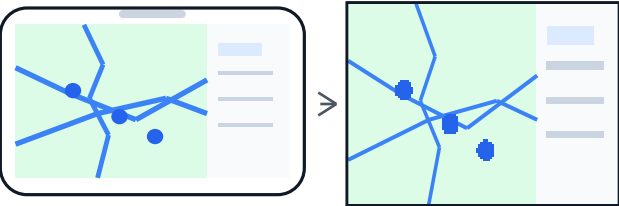
2.9 Tablet Mockups and Layout Downsampling Q&A Contract

Large-screen tablet layouts use their own portrait and landscape matrix dimensions, keeping the physical aspect ratio instead of forcing every replay into a phone grid.

Tablet Portrait Split Content -> 96x128



Tablet Landscape Map -> 128x96



Tablet matrices preserve master-detail layouts, side rails, and split-pane content without collapsing coordinates. Training can pad batches to 128x128 and use valid masks.

Layout and Downsampling Q&A Contract

Question	Answer in the current contract
Should every replay be one dimension?	No. Train with padded 128x128 batches and valid masks, while preserving native grid dimensions per sample.
How does the worker know landscape?	It derives orientation from dimensions: width > height means landscape. No special SDK flag is required.
What if an event lacks dimensions?	Touch rows use the rolling viewport fallback. Default fallback remains phone portrait 64x128.
Are touch coordinates quantized?	Yes. x_norm_bucket/y_norm_bucket use 0..1000 buckets with step 5, plus exact grid cell indices.
How are web DOM replays handled?	Web rweb rows are DOM event streams, not mobile raster matrices. They use a separate DOM encoder.
What if hierarchy cadence differs by SDK?	manifest.capture_profile.hierarchy records observed cadence, alignment ratio, and coverage ratio per sample.

2.10 Hierarchy Capture Cadence Profile

Different SDK/package versions may capture native view hierarchy every visual frame, every other frame, every one second, or every two seconds. Newer mobile SDK builds trigger hierarchy capture from screenshot capture using the same frame timestamp. The model should not infer this from SDK version; the export detects it from timestamps, ratios, and nearest screenshot deltas. Good alignment does not require 100% perfection: 80% or better frame coverage/alignment is treated as usable because real devices can drop frames, dedupe uploads, or jitter timestamps.

manifest.json / quality.json fields

```
"capture_profile": {
  "hierarchy": {
    "present": true,
    "cadence_mode": "per_visual_frame",
    "configured_interval_ms": null,
    "observed_median_interval_ms": 500,
    "observed_snapshot_count": 206,
    "screenshot_frame_count": 240,
    "hierarchy_to_screenshot_ratio": 0.86,
    "hierarchy_screenshot_alignment_ratio": 0.98,
    "screenshot_hierarchy_coverage_ratio": 0.86,
    "alignment_threshold_ratio": 0.8,
    "alignment_tolerance_ms": 500,
    "nearest_screenshot_median_delta_ms": 0,
    "alignment": "screenshot_frame_aligned"
  }
}
```

cadence_mode	Meaning	Training treatment
per_visual_frame	Hierarchy covers >=80% of screenshot frames and hierarchy rows are near screenshots.	Strongly pair hierarchy rows with screenshot frames; no 100% requirement.
every_other_visual_frame	Hierarchy appears on roughly every second visual frame and >=80% of hierarchy rows align.	Still frame-aligned; pair to nearest screenshot and use a small temporal mask.
fixed_interval	Hierarchy appears every fixed interval, commonly 1000-2000 ms.	Treat as sparse auxiliary context; do not assume every screenshot has a matching tree.
sparse_interval / single_snapshot	Few hierarchy samples or irregular timestamps.	Downweight hierarchy features; use matrix + events as primary evidence.
absent	No usable hierarchy snapshots.	Train from screenshot matrix, rrweb, and/or events only.

quality.json warnings

- hierarchy_sparse_timeline: cadence is fixed_interval, sparse_interval, or single_snapshot.
- hierarchy_not_frame_aligned: hierarchy should not be joined as one-to-one with screenshot frames.
- No warning for per_visual_frame or every_other_visual_frame when alignment ratios are >=0.8 and nearest screenshot deltas remain small; this is the good hierarchy path for the new SDK frame-triggered capture.

2.11 Masking, Keyboard, and Special Surface Contract

Research-lake masking is handled as a privacy and training-quality profile, not as raw mask geometry. Native screenshots are already redacted before JPEG upload. Native hierarchy rows preserve only coarse mask flags such as `is_masked` and `is_keyboard_or_system`. Web `rrweb` rows are sanitized before recording, then exported with masked DOM flags when the sanitized event exposes them.

manifest.json / quality.json masking fields

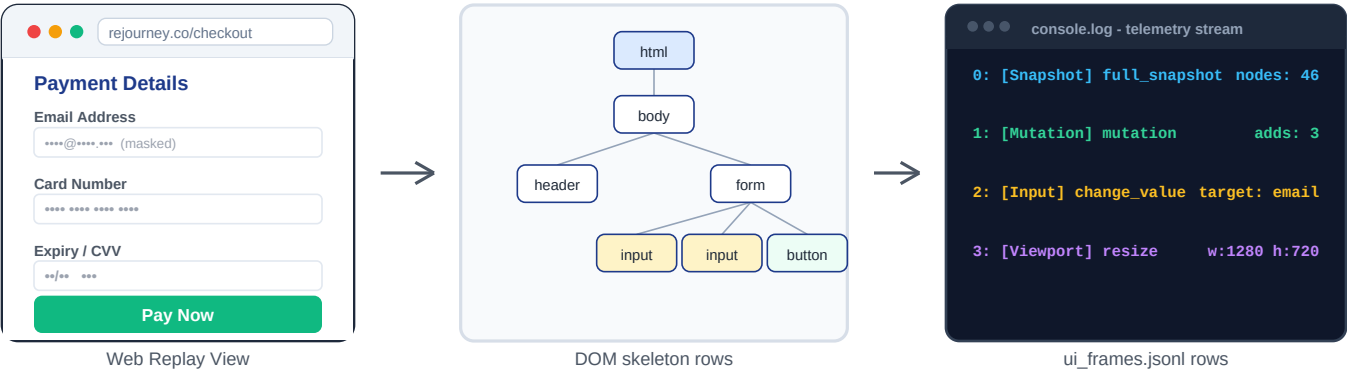
```
"capture_profile": {
  "masking": {
    "text_input_masking_policy": "all",
    "image_video_masking_policy": "none",
    "screenshot_pixels_post_redaction": true,
    "hierarchy_mask_observed": true,
    "hierarchy_masked_element_count": 12,
    "hierarchy_masked_input_count": 5,
    "hierarchy_keyboard_or_system_element_count": 0,
    "hierarchy_keyboard_or_system_policy": "exclude_or_downweight",
    "rrweb_mask_observed": true,
    "rrweb_masked_input_value_count": 8,
    "rrweb_masked_media_attribute_count": 2
  }
}
```

Surface / signal	Research-lake handling	Training treatment
Native <Mask> / explicit occlusion	SDK sets <code>masked=true</code> ; export keeps <code>is_masked</code> and counts only.	Suppress text/content learning; use <code>role/layout</code> only.
Text inputs	Policy is <code>all</code> or <code>secure_only</code> ; secure fields are always sensitive.	<code>secure_only</code> is a warning because ordinary text inputs may remain visible by policy.
Images/videos/camera	Screenshot pixels are redacted by SDK when policy/explicit mask applies.	Do not infer private media content from masked regions.
Keyboard/system windows	iOS scanner skips them; if nodes appear, export marks <code>is_keyboard_or_system</code> .	Exclude or downweight; never treat as app content.
Web <code>rrweb</code> masks	Input/media attributes sanitized to <code>***</code> or blocked via <code>rrweb</code> selectors.	Use DOM <code>role/depth/action</code> , not raw text/media.

3.1 Web DOM Visual Examples

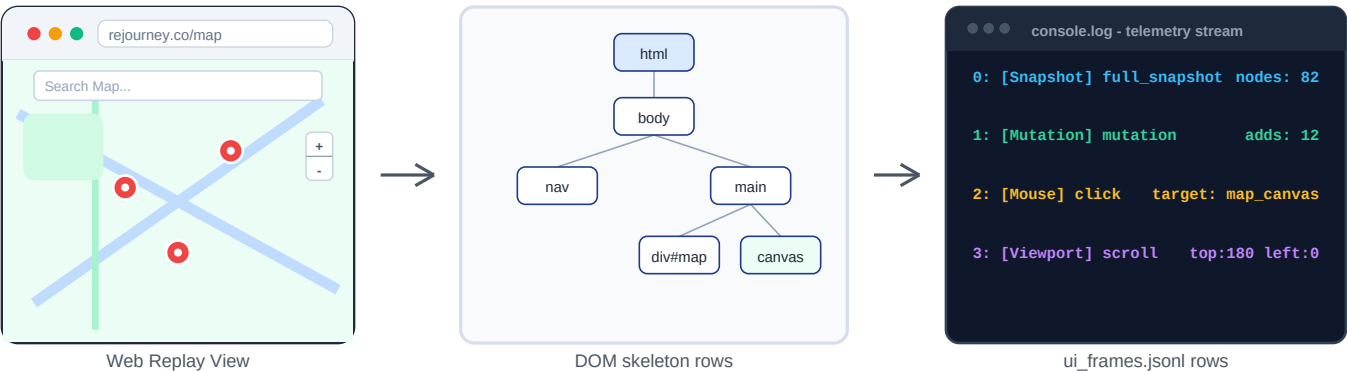
Web replay examples are shown as browser UI to DOM tree to rrweb event rows. This is deliberately different from the mobile UI matrix examples: the web encoder consumes semantic tree and mutation events, not a 64x128 raster matrix.

Web Checkout/Form Replay -> DOM Tree -> rrweb Rows



Inputs are represented as DOM roles and masked attribute signals. The model can learn form structure, submit controls, and action order without seeing raw field values.

Web Map/Canvas Replay -> DOM Shell -> rrweb Rows



Mapbox, Google Maps, Apple Maps web embeds, and canvas-heavy screens expose DOM shells and actions, but not all rendered pins or tiles. Treat DOM as semantic context; use events for object interactions and consider a future privacy-safe raster lane for canvas-heavy web apps.

3.2 Web DOM rrweb Replay: Stream Contract

Web replay data is intentionally not converted into a 64x128 or tablet raster grid. The web lane keeps rrweb-compatible DOM events: full snapshots establish the DOM baseline, incremental events update that baseline, and meta/resize/scroll events preserve the viewport context needed to replay or model the page. This matches web-agent datasets such as Mind2Web and WebLINX at the semantic level (HTML/DOM plus actions), while remaining a session-replay stream rather than a static page corpus.

Field / signal	Current contract
source_kind=rrweb	Routes rows to rrweb_dom_encoder, never screenshot_grid_encoder.
rrweb_event_kind	String family: full_snapshot, mutation, meta, scroll, viewport_resize, incremental, event.
capture_profile.rrweb	Quality summary: replay_basis, full_snapshot_count, mutation_count, DOM skeleton count, viewport/page missing counts.
ui_skeleton rows	Masked DOM tree nodes extracted from full snapshots and mutation-added nodes.

Correct rrweb Mutation Row Shape

```
{
  "source_kind": "rrweb",
  "visual_modality": "dom_event_stream",
  "recommended_encoder": "rrweb_dom_encoder",
  "rrweb_event_type": 3,
  "rrweb_incremental_source": 0,
  "rrweb_event_kind": "mutation",
  "mutation_adds_bucket": 2,
  "mutation_removes_bucket": 0,
  "viewport_width_bucket": 1280,
  "viewport_height_bucket": 720,
  "page_width_bucket": 1280,
  "page_height_bucket": 2048,
  "text_class": "masked"
}
```

Replay Quality Warnings

- rrweb_missing_full_snapshot: incremental rows exist without a DOM baseline; train as partial event telemetry, not replay-grade DOM.
- rrweb_incremental_without_full_snapshot: mutation stream cannot be reconstructed without prior state.
- rrweb_viewport_dimensions_missing / rrweb_page_dimensions_missing: layout scale is incomplete.
- rrweb_full_snapshot_without_dom_skeleton: snapshot existed but did not produce modelable DOM nodes.

3.3 HTML Tag-to-Role Mappings & Privacy Flags

The DOM lane keeps semantic structure while stripping user-readable content. This is closer to Mind2Web/WebLINX-style HTML observations than to mobile screenshot datasets, but it is more privacy-constrained: selectors, URLs, raw text, and asset sources are never model inputs. The model sees roles, depth, child-count buckets, and coarse attribute-presence flags:

HTML Tag / Context	Mapped Role	Privacy-Preserving Attribute Flags
button, a, input[type=submit]	control	has_class: true if custom CSS class names exist.
input, textarea, select	input	has_href: true if hyperlink target is populated.
img, svg, video, canvas	media	has_src: true if asset source URL is present.
h1, h2, h3, h4	heading	has_alt: true if accessibility alt text is set.
p, span, text nodes	text	has_placeholder: true if placeholder text exists.

Audit Notes

- Strong point: DOM depth and role sequence make web form funnels, checkout flows, and navigation menus modelable without screenshots.
- Weak point: visual styling, occlusion, z-index, and canvas/map contents are only weakly represented unless the DOM exposes useful attributes.
- Compatibility point: web DOM samples can pretrain semantic action encoders, but they should not be mixed with mobile raster grids before late fusion.

3.4 Web DOM Replay vs. Mobile UI: Dual-Encoder Requirements

The critical training rule is that web DOM rows and mobile screenshot rows are not the same tensor family. Mobile UI frames are fixed-shape visual matrices; web rows are temporal graph/tree events. A single early encoder would either destroy mobile visual locality or flatten away DOM hierarchy. The correct path is modality-specific encoders followed by late fusion:

Feature / Modality	Mobile UI Modality	Web DOM Modality
Data Representation	Luma/Edge/Color Grids (Raster)	rrweb DOM Mutation Streams (JSON tree)
Ingested File	ui_frames.jsonl.gz (matrices)	ui_frames.jsonl.gz (JSON elements)
Encoder Family	screenshot_grid_encoder (CNN)	rrweb_dom_encoder (GNN/Transformer)
Interaction Mapping	Quantized 2D grid cells	HTML tag-to-role pointers
Structural Scope	Visual viewport raster	Full nested DOM tree structures

Research Compatibility Matrix

Resource shape	What transfers	What does not transfer
Rico / MobileViews	Mobile screenshot + hierarchy pretraining ideas.	No web DOM mutation timeline.
Mind2Web / WebLINX	HTML/DOM/action supervision and web-agent action prediction.	Not native mobile grids; not private replay rows by default.
WebArena	Agent evaluation tasks over realistic web environments.	Benchmark sites, not production session replay schema.
rrweb	Snapshot and incremental DOM replay mechanics.	No ML labels or business outcomes by itself.

During sequence-to-sequence pretraining, web and mobile sessions must be routed through their respective encoders. Only latent embeddings, labels, and calibrated behavioral outcomes should be joined across modalities.

4.1 Behavioral Outcomes Dataset: Lane Ingestion

The behavioral outcomes dataset captures session clickstreams, performance parameters, and conversions for bounce, observe-only, or quota-limited visual recordings. All visual frame files are completely omitted from this dataset.

Mitigating Model Selection Bias

Training predictive models solely on interaction sessions creates a severe selection bias, as sessions with visual recordings skew heavily toward highly active, long-duration users. Including behavioral outcomes sessions calibrates propensity models and establishes true session base rates:

- Calibrates baseline conversion rates across bounces and observe-only traffic.
- Provides true distributions of session durations and total event counts.
- Ensures models learn patterns of low-engagement users who churn early.
- Prevents overestimating lifetime value (LTV) on low-activity client segments.

4.2 Behavioral Outcomes Dataset: Session Signals and Labels

The behavioral outcomes lane is not only funnel or churn. Every exported sample carries manifest, quality, session_metrics, labels, and event rows when events are present. That makes it useful for engagement, UX friction, reliability, network, SDK health, cohort drift, and sequence modeling even when the session has no visual replay.

Every exported behavioral sample can support these tasks

signal family	stored fields / files	training and research use
Context and cohort	manifest.json: platform, app_version_bucket, sdk_version_bucket, duration_seconds_bucket, retention_days, geo, source.reason, quality_tier	Cohort controls, drift checks, source-bias analysis, and model reweighting.
Screen journey	visitor_context: is_bounced, screens_visited_count, screen_path_keys	Path embeddings, bounce prediction, onboarding drop-off, and navigation similarity.
Activity intensity	session_metrics.json: total_events, custom_event_count, touch_count, scroll_count, gesture_count, input_count	Engagement scoring, session segmentation, and active-vs-passive usage modeling.
UX friction	rage_tap_count, dead_tap_count, coordinate_missing_count, viewport_source_counts	Frustration labels, broken target detection, and noisy-coordinate filtering.
Network and API	api_total_count, api_success_count, api_error_count, api_avg_response_ms_bucket, network_type, cellular_generation, is_constrained, is_expensive	Latency/failure prediction, bad-network controls, and reliability-aware outcome analysis.
Stability	crash_count, anr_count, error_count	Crash/error outcome labels and exclusion rules for sessions interrupted by failures.
SDK delivery health	sdk_upload_success/failure/retry counts, circuit breaker, memory eviction, offline persist, upload duration, bytes uploaded/evicted	Data-quality weights, missingness audits, and capture-system health monitoring.
Event sequence	events.jsonl.gz: elapsed_ms_bucket, event_family, event_kind, event_name_key, target_key, x/y buckets, grid cells, shape/product/plan/price keys	Next-action models, temporal behavior embeddings, heatmaps, and custom-event consistency checks.
Supervised labels	labels.json: conversion, revenue bucket, lifecycle_events_present, max_funnel_stage_reached, API/stability/fat-finger labels, abandonment flags	Multi-task heads beyond churn: conversion, reliability, frustration, and lifecycle outcomes.

Custom event names still matter, but they are one label layer: recognized names populate funnel_transition and lifecycle_events_present. They should be used beside the session, reliability, network, SDK-health, and UX-friction signals above, not as the whole dataset definition.

4.3 ML Labels and Conversion Label Facts

Labels and target outcomes are stored in labels.json and compacted into the training_labels database partition. These label facts serve as regression and classification targets for cohort modeling:

Target Category	Key Columns / Fields	Description / Value Ranges
Funnel Completion	max_funnel_stage	Integer (0=landing, 1=signup, 2=cart, 3=checkout, 4=purchase)
Conversion Label	is_conversion_session	Boolean (true if purchase event was completed)
Revenue Outcome	revenue_bucket	Quantized nearest \$10 up to \$200, \$50 up to \$1,000
Retention / Churn	retention_status	Active status evaluated at Day 1, Day 7, Day 14, and Day 30
Modality Stability	quality_tier	Quality tier gate (pass, fail, warning)

These targets are aligned during training with user activity timelines, calibrating the conversion propensities predicted by sequence-to-sequence networks.

5.1 Curated Lake: Parquet ML Contract

The curated layer is not just a storage optimization. It is the contract that prevents training jobs from mixing incompatible modalities. Raw JSONL files preserve maximum detail; curated Parquet tables expose stable row families, partition keys, and quality gates for reproducible model training. Athena/AWS guidance favors columnar formats such as Parquet and partition keys that match query filters; for ML, the same idea becomes: route by source_lake, table, date, platform, modality, and quality profile.

Curated Invariants

- One schema v1 contract: no legacy grid options, no alternate-schema language, and no hidden SDK-version assumptions.
- No early modality merge: screenshot grids, native hierarchy snapshots, rweb DOM streams, and behavioral-only rows stay in separate fact tables until model-specific readers join them.
- Quality-first filtering: quality_fact exposes hierarchy cadence and rweb replay basis so jobs can choose replay-grade samples, sparse-context samples, or tabular-only samples deliberately.
- Privacy boundary stays intact: raw screenshots, raw DOM text, URLs, selectors, and identifiers are absent before compaction and remain absent afterward.

Partition and Table Shape

```
v1_curated/  
source_lake=interaction/table=session_fact/date={dt}/platform={platform}/  
source_lake=interaction/table=quality_fact/date={dt}/platform={platform}/  
source_lake=interaction/table=event_fact/date={dt}/event_family={family}/  
source_lake=interaction/table=ui_frame_fact/date={dt}/platform={platform}/  
source_lake=interaction/table=ui_skeleton_fact/date={dt}/platform={platform}/  
source_lake=behavioral_outcomes/table=training_labels/date={dt}/platform={platform}/  
source_lake=combined/table=event_fact/date={dt}/event_family={family}/
```

5.2 Curated Training Reader, Tables and Quality Gates

The curated Parquet layer is a multi-table training contract, not just a funnel table. The compactor preserves worker outputs and emits session, event, label, quality, visual, skeleton, network, and stability facts so training code can build visual, sequence, reliability, and quality-aware datasets from the same sample keys.

Curated Parquet tables and model uses

signal family	stored fields / files	training and research use
session_fact	source_lake, project/sample/date, platform, app/sdk buckets, duration, retention, source_reason, bounce, screen counts, event/API/stability counts	Session embeddings, cohort balancing, sample weighting, and drift analysis.
event_fact	event_index, elapsed_ms_bucket, event_family, event_kind, funnel_transition, screen_key, target_key, coordinates, grid cells, business keys, event_shape_key	Sequence models, next action, heatmaps, custom-event hygiene, and lifecycle labels.
ui_frame_fact	screenshot luma/edge/color grids, hierarchy frame summaries, rweb event rows, grid shape, source_kind, recommended_encoder, delta signatures	Visual UI encoders, layout clustering, DOM replay encoders, and state-change models.
ui_skeleton_fact	element role, source_kind, type_family/tag_class, depth, child counts, x/y cells, mask flags, keyboard/system flags, media/map/webview roles	Element-role learning, accessibility/layout structure, keyboard filtering, and surface-aware modeling.
training_labels	conversion/revenue, max funnel stage, lifecycle_events_present, purchased products, API/stability/fat-finger labels, abandonment flags	Multi-task supervision for outcomes, reliability, frustration, and lifecycle progression.
quality_fact	quality_tier, modality counts, grid shapes, viewport missingness, hierarchy cadence/alignment, rweb replay basis, masking policy, warnings	Eligibility filters, modality masks, loss weighting, and privacy/capture audits.
network_fact	API totals/success/errors, avg response bucket, network_type, cellular_generation, constrained/expensive flags	Network-aware performance models and confounder controls for poor connectivity.
stability_fact	crash_count, anr_count, error_count, rage_tap_count, dead_tap_count, has_stability_failure	Failure analysis, exclusion windows, and UX/stability outcome heads.
combined outputs	source_lake=combined keeps shared session/event/label columns across interaction and behavioral_outcomes	Common training splits across mobile replay, web DOM replay, and metrics-only sessions.

The training reader enforces modality-specific safety: visual grids must match grid_columns x grid_rows across luma, edge, and color arrays; mobile hierarchy rows must carry sparse/cadence/alignment fields; rweb rows must use known replay event kinds. Funnel transitions remain useful, but quality_fact and table-specific masks decide whether a sample is fit for a given model head.

6.1 Tabular Machine Learning Model Architectures

Tabular Models (XGBoost / LightGBM)

Objective: Train tabular models on session-level aggregates for customer lifetime value (LTV) regression and cohort churn prediction:

- Tabular inputs (derived from session_fact): country, app_version_bucket, sdk_version_bucket, total_events, touch_count, dead_taps, rage_taps, crash_count, error_count, and network transport latency.
- ML targets (from training_labels): is_conversion_session (0 or 1) and conversion_revenue_bucket (\$0..\$1,000).
- Loss functions: Binary log-loss for conversion prediction, and Tweedie loss for right-skewed LTV regression.

Feature Importance Rankings

Feature relevance evaluations show that dead taps, rage taps, and connection latency are strong predictors of early checkout abandonment.

6.2 Sequential Machine Learning Model Architectures

Sequence-to-Sequence Models (LSTM / Transformers)

Objective: Model the exact temporal sequence of user actions in clickstream logs to predict next-action gestures and path-based conversions:

- Sequential inputs (from event_fact & ui_skeleton_fact): Action sequence [a_1, a_2, ..., a_t] (kind, transition, elapsed_ms_bucket) and layout sequence [S_1, S_2, ..., S_t] (bounding box structures, semantic element roles).
- Loss functions: next-action cross-entropy loss and path-based conversion propensity loss.
- Architecture: Multi-headed self-attention layer followed by causal action decoders and conversion prediction heads.

Handling Missing Modalities

For sessions in the behavioral outcomes dataset that lack visual grids, the layout input sequence [S_1, ..., S_t] is padded with a special masked token, bypassing the screenshot encoder.

7.1 Compatibility, Fixes & Appendix

Resource	Compatibility with this export
Rico-style mobile UI corpora	Screenshot + view-hierarchy shape is compatible with our mobile lane, but our raster grids are lower fidelity and need hierarchy cadence gates.
Enrico / curated mobile screens	Useful for visual pretraining ideas, but production sessions need outcomes, timestamps, and privacy masking.
MobileViews-like UI pairs	Large screenshot/hierarchy pairs transfer conceptually; our tablet/landscape shapes and sparse hierarchy profile are stricter.
Mind2Web / WebLINX	Compatible with web action and DOM/HTML modeling ideas; not shape-compatible with mobile raster grids.
rrweb	Directly compatible with session replay mechanics: full snapshots plus incremental DOM events.
AWS/Athena/Parquet guidance	Supports the curated design: columnar Parquet, query-aligned partitions, and compacted files.

Implemented Fixes & Recommendations

- Orientation-aware layout: aspect ratio rules derived from dimensions prevent visual distortion.
- DOM replay quality: capture_profile.rrweb records replay_basis, full-snapshot coverage, mutation counts, and viewport/page coverage.
- Masking profile: capture_profile.masking records configured text/media masking, observed native/web masks, and keyboard/system counts.
- Curated gating: quality_fact exposes hierarchy cadence and rrweb replay fields for training filters.
- PII safety: DOM text masking, coordinates nullification, and k-anonymity city thresholding.
- Outcomes calibration: including behavioral clickstream data resolves visual selection bias.

Appendix A: Active Schema v1 Contract

```
manifest.capture_profile.hierarchy -> cadence_mode, observed_median_interval_ms, alignment,
  hierarchy_screenshot_alignment_ratio, screenshot_hierarchy_coverage_ratio,
  alignment_threshold_ratio
manifest.capture_profile.rrweb -> replay_basis, full_snapshot_count, mutation_count,
  viewport_missing_count
manifest.capture_profile.masking -> text_input_masking_policy, image_video_masking_policy,
  masked counts, keyboard/system counts
source_kind=screenshots -> visual_modality=raster_feature_grid (grid_columns, grid_rows,
  luma_grid, edge_grid, color_grid)
source_kind=hierarchy -> visual_modality=mobile_hierarchy_snapshot
(hierarchy_snapshot_sparse boolean, cadence/alignment, is_masked, is_keyboard_or_system)
source_kind=rrweb -> visual_modality=dom_event_stream (rrweb_event_kind string, mutation
  buckets, page/viewport buckets)
```

7.2 External Research Comparison: Similar Data and Uses

This audit looked at public UI, web-agent, ecommerce clickstream, and session-replay work that stores comparable combinations of screenshots, hierarchy/DOM structure, actions, event sequences, and outcomes. The main lesson is that our lake should stay multi-modal and multi-task: funnel labels are only one use, while visual structure, element semantics, temporal behavior, reliability, network, and friction signals are separate model targets and controls.

Comparable datasets and transferable lessons

source	what they collected	how they used it	lesson for our lake
Rico	66k+ Android UI screens from 9.3k apps; screenshots, augmented view hierarchies, UI elements, gestures, traces, animations, semantic labels.	Layout autoencoders, UI similarity search, layout/code generation, user interaction modeling, design-performance/perception prediction.	Keep pixels + hierarchy + events together; hierarchy is what gives boxes roles and avoids pure OCR/image noise.
Enrico, VINS, Screen2Words, Widget Captioning	Curated RICO-derived screens, view hierarchies, semantic wireframes, screen summaries, and widget captions.	Topic modeling, visual UI search, screen summarization, widget/accessibility captioning, element-function learning.	Curated labels and element roles matter. Add high-quality subsets and avoid training only on noisy raw exports.
Android in the Wild	Human demonstrations: Android screenshots, UI annotation boxes/text/types, action type, touch/lift coordinates, device/API metadata.	Mobile device-control imitation learning and evaluation across tasks, devices, Android versions, and screen resolutions.	Our grid cells are compatible with action modeling, but keep viewport buckets, form factor, orientation, and sequence order.
Mind2Web, WebLINX, WebArena, WebVoyager	Web tasks with DOM/HTML, screenshots, action history, browser traces, and realistic websites or benchmark environments.	Generalist web agents, next-action prediction, element ranking/pruning, multimodal browser control, cross-site generalization.	DOM replay should use a different encoder than mobile grids; prune/compact DOM but keep screenshot/action history alignment.
RetailRocket, YOOCHOOSE, Coveo SIGIR eCom	Session clickstreams: views/detail events, add-to-cart, purchase/transaction, timestamps, product/catalog/query metadata.	Purchase intent, session-based recommendation, cart prediction, abandonment, ranking, and customer-intent modeling.	Behavioral_outcomes is valuable even without replay: event order, elapsed buckets, product keys, and labels are model-ready.
rrweb and session replay products	DOM snapshots plus timestamped mutations/events; tools such as Clarity/PostHog use replay, heatmaps, rage/dead clicks, network/error context.	Debug UI issues, support, heatmaps, friction detection, funnel replay, performance context, and behavior segmentation.	Quality/masking gates are essential. Keep rrweb full snapshots, viewport/page coverage, dead/rage taps, network, and privacy facts.

Sources

Rico: interactionmining.org/rico | Android in the Wild: github.com/google-research/google-research/tree/master/android_in_the_wild | Mind2Web: osu-nlp-group.github.io/Mind2Web | WebLINX: mcgill-nlp.github.io/weblinx | WebArena: webarena.dev | WebVoyager: aclanthology.org/2024.acl-long.371 | RetailRocket/YOOCHOOSE/Coveo: [Kaggle/RecSys/SIGIR eCom dataset pages](https://kaggle.com/datasets/recsys-sigir/eCom) | rrweb: github.com/rrweb-io/rrweb | Clarity/PostHog docs for replay and friction metrics.

7.3 Aggregate Revenue Outcomes Lane

The research lake now has a third additive lane for external provider revenue trends. This lane exists for UX and revenue correlation research when revenue is known only at a project/provider/day grain, such as RevenueCat or Superwall rollups. It must not be interpreted as session conversion attribution.

Design contract

- Store aggregate growth and decline signals over time, not per-session purchase outcomes.
- Keep the existing session lanes unchanged: interaction and behavioral_outcomes still own session facts, events, quality facts, and training labels.
- Do not copy provider API keys, connection IDs, customer IDs, subscriber IDs, transaction IDs, subscription IDs, refund IDs, session IDs, device IDs, user identifiers, or hashes of those values into the research lake.
- The only join dimension is the lake-standard anonymous project_key plus date/provider/currency. Training joins must be cohort/time-window joins, never individual session conversion joins.

Raw landing zone

Revenue outcome files are written as idempotent project/provider/currency/date objects:

```
v1/lake=revenue_outcomes/project_key={project_hash}/date={yyyy-mm-dd}/provider={provider}/currency={currency}/
manifest.json
quality.json
daily_revenue.json
v1/lake=revenue_outcomes/project_key={project_hash}/date={yyyy-mm-dd}/provider={provider}/currency={currency}.zip
```

The path intentionally has no sample_key and no provider row identifier. Re-exporting the same date overwrites the same aggregate object, so provider corrections update the project-day fact without creating transaction-like lake records.

Field group	Stored examples	Meaning
Grain and attribution	provider, currency, sample_date, attribution_scope=project_day, session_attribution_available=false	Declares that the fact is aggregate-only and cannot label individual sessions.
Revenue buckets	gross_revenue_bucket, refund_revenue_bucket, fee_revenue_bucket, net_revenue_abs_bucket, net_revenue_direction	Coarse money buckets preserve trend shape without exporting raw provider facts.
Counts	transaction_count_bucket, refund_count_bucket, subscriber_count_bucket, trial_count_bucket, subscription_start_count_bucket, cancellation_count_bucket, conversion_count_bucket	Provider-side activity volume as bucketed project-day controls.
Trend features	previous_day_net_revenue_abs_bucket, net_revenue_delta_direction, trailing_7d_net_revenue_delta_abs_bucket	Growth/decline supervision for cohort and UX correlation models.

Curated Parquet Contract

The compactor emits revenue outcomes into a separate curated table. It does not merge these rows into training_labels, because the row is not a session conversion label.

```
v1_curated/source_lake=revenue_outcomes/table=daily_revenue_fact/date={dt}/provider={provider}/currency={currency}/
```

Training usage

- Allowed: join session or UX aggregates to revenue_outcomes by project_key and time windows, for example UX on day D to revenue D+1, D+7, or D+28.
- Allowed: use provider, currency, platform/app-version cohorts from session_fact, and quality_fact controls to model revenue growth or decline.
- Not allowed: set is_conversion_session or conversion_revenue_bucket on a session from this lane.
- Not allowed: infer that a specific replay, user, device, or session converted because project-day revenue increased.

Operational migration behavior

- The exporter reads project_revenue_daily, builds anonymized aggregate documents, and writes them under lake=revenue_outcomes.
- A checkpoint table records backfill and incremental cursors, so existing research-lake data does not need to be restarted or rewritten.
- Initial deploy backfills bounded batches over successive worker cycles, then switches to updated_at incremental export.
- Quality files mark these facts as aggregate_only with a not_session_attributed warning.

Privacy boundary

This lane exports only coarse aggregate facts. Raw amounts are bucketed. Counts are bucketed. Deltas and trailing-window totals are bucketed and direction-coded. No raw provider metadata or stable provider object identifiers are exported, even as hashes. The lane may carry provider and currency names because they are analytic dimensions, not customer or transaction identities.

Example row shape

```
{ "lake": "revenue_outcomes", "project_key": "...", "sample_date": "2026-06-11", "provider": "revenuecat",  
  "currency": "usd", "attribution_scope": "project_day", "session_attribution_available": false,  
  "gross_revenue_bucket": 150, "refund_revenue_bucket": 10, "net_revenue_abs_bucket": 140, "net_revenue_direction":  
  "positive", "transaction_count_bucket": 10, "net_revenue_delta_direction": "increase",  
  "trailing_7d_net_revenue_delta_abs_bucket": 200 }
```